Highlights

The Beggining of Control Revolution: Ofset-free Koopman MPC

Patrik Valábek

- Research highlight 1
- Research highlight 2

The Beggining of Control Revolution: Ofset-free Koopman MPC

Patrik Valábek

[a]*STU, Redlinskeho 9, Bratislava, Slovakia*

Abstract

Keywords:

1. Introduction

2. Preliminaries and notation

In this section, we present the preliminary concepts used throughout this paper, namely the offset-free model predictive control (MPC) framework and the Koopman operator theory.

2.1. Offset-free MPC

In this subsection, we present the standard offset-free MPC framework introduced by Pannocchia and Rawlings (2003). The framework consists of three parts, namely a state observer, a target optimization, and a dynamic optimization. Together they introduce an integral action into the MPC, which mitigates the process-model mismatch and ensures offset-free control in the steady state. Without this compensation, the closed loop does not reach a reference other than the origin, because the predictions are made with a model that differs from the plant.

The offset-free augmented system can be written in the following state-space form, including the disturbance input matrix B_d and disturbance output matrix C_d:

$$\begin{bmatrix} x_{k+1} \\ d_{k+1} \end{bmatrix} = \begin{bmatrix} A & B_d \\ 0 & I \end{bmatrix} \begin{bmatrix} x_k \\ d_k \end{bmatrix} + \begin{bmatrix} B \\ 0 \end{bmatrix} u_k, \qquad y_k = \begin{bmatrix} C & C_d \end{bmatrix} \begin{bmatrix} x_k \\ d_k \end{bmatrix}, \tag{1}$$

where x_k is the original system state, d_k is the estimated constant disturbance, u_k is the control input, y_k is the output, A, B, and C are the original

state-space matrices, B_d and C_d encode how the disturbance enters the state and output equations, and I is the identity matrix. Here, n_x, n_u, n_y, and n_d denote the state, input, output, and disturbance dimensions, respectively, so that $x_k \in \mathbb{R}^{n_x}$, $u_k \in \mathbb{R}^{n_u}$, $y_k \in \mathbb{R}^{n_y}$, $d_k \in \mathbb{R}^{n_d}$, $A \in \mathbb{R}^{n_x \times n_x}$, $B \in \mathbb{R}^{n_x \times n_u}$, $C \in \mathbb{R}^{n_y \times n_x}$, $B_d \in \mathbb{R}^{n_x \times n_d}$, $C_d \in \mathbb{R}^{n_y \times n_d}$, and $I \in \mathbb{R}^{n_d \times n_d}$.

For offset-free behavior, the number of disturbances n_d must be less than or equal to the number of measurements n_y. Note that offset-free tracking is achieved in at most as many outputs as there are estimated disturbances. For the case $n_d = n_y$, the choice of the matrices B_d and C_d has to satisfy the necessary condition

$$\operatorname{rank} \begin{bmatrix} I - A & -B_d \\ C & C_d \end{bmatrix} = n_x + n_y. \tag{2}$$

Neither the state nor the disturbance is measured, and both are therefore reconstructed by the observer before the two optimization problems are solved at each sampling instant.

2.2. Koopman operator

The Koopman operator theory was introduced by Koopman (1931) and popularized by Mezić (2005). The theory represents a nonlinear system exactly by a linear operator, by lifting its states or measurements into an infinite-dimensional space in which the lifted variables evolve linearly. TODO: Intro?

Consider the nonlinear discrete-time system with control

$$x_{k+1} = f(x_k, u_k), \tag{3}$$

where $f : \mathbb{R}^{n_x} \times \mathbb{R}^{n_u} \to \mathbb{R}^{n_x}$ is the nonlinear transition map of the system. For such a system the Koopman operator $\mathcal{K}$ is defined on an extended state space that carries the whole sequence of the future inputs (Korda and Mezić, 2018), and it advances the lifting functions, also called the observables, by one step along the trajectories of (3), which is written as

$$(\mathcal{K}\psi)\,(x_k,\, \mathbf{u}_k) = \psi(f(x_k,\, \mathbf{u}_k(0))\,,\, \mathcal{S}\mathbf{u}_k)\,, \quad \forall \psi \in \mathcal{F}, \tag{4}$$

where $\psi : \mathbb{R}^{n_x} \times \ell(\mathcal{U}) \to \mathbb{R}$ is a scalar lifting function, $\mathcal{F}$ is the space of the lifting functions on which $\mathcal{K}$ is defined, $\mathbf{u}_k \in \ell(\mathcal{U})$ is the sequence of the inputs applied from the step k onwards, $\ell(\mathcal{U})$ is the space of such sequences

taking values in the input constraint set $\mathcal{U}$, $\mathbf{u}_k(0)$ is the input applied at the step k, and $\mathcal{S}$ is the left shift operator defined by $(\mathcal{S}\mathbf{u}_k)(i) = \mathbf{u}_k(i+1)$. The operator $\mathcal{K}$ is linear even though the map f is not, and this linearity is obtained at the price of an infinite-dimensional $\mathcal{F}$ and of a state space that carries the entire input sequence.

Any practical use of (4) therefore restricts $\mathcal{F}$ to the span of n_z lifting functions of the state alone, which are collected in a single vector-valued map, giving

$$z_k = g(x_k) = [\psi_1(x_k),\ \ldots,\ \psi_{n_z}(x_k)]^\intercal, \tag{5}$$

where g is the vector-valued lifting map that collects the n_z retained lifting functions and $z_k \in \mathbb{R}^{n_z}$ is the lifted state. Under this restriction, and with the dependence on the input assumed to be affine, the operator $\mathcal{K}$ reduces to a finite-dimensional linear model in the lifted coordinates that is driven by the input through a constant matrix, which is the structure stated in Sec. 3 and the one that keeps the MPC problem a quadratic program. The model is exact only if the span of $\psi_1, \ldots, \psi_{n_z}$ is invariant under $\mathcal{K}$, so the choice of the lifting functions determines its accuracy, which is why they are identified from data in this work rather than picked from a fixed library. The state x_k of the controlled plant is not measured here, and the lifting map is consequently evaluated at the measured output, $z_k = g(y_k)$ with $g : \mathbb{R}^{n_y} \to \mathbb{R}^{n_z}$.

Over the years, many data-driven approximations of the Koopman operator have been proposed, starting with the connection between the dynamic mode decomposition (DMD) and the Koopman operator theory established by Rowley et al. (2009) and extended to the extended DMD (EDMD) by Williams et al. (2015). More recently, neural networks have been used to approximate the lifting functions, so that the lifted system evolves linearly. These algorithms were developed not only for autonomous systems but also for systems with control (Kaiser et al., 2021). The Koopman operator was brought together with the MPC by Korda and Mezić (2018), and in recent years almost every kind of MPC has been reformulated to accept an approximation of the Koopman operator, mostly represented as a linear time-invariant (LTI) system in the state-space form. The offset-free version is no exception. Chen et al. (2022) applied the offset-free control to a soft robot, while Pan et al. (2024) applied the same methodology to thermal regulation, which is an application closer to the process control.

TODO: Intro?

For the purpose of this paper, we use the deep Koopman approximation

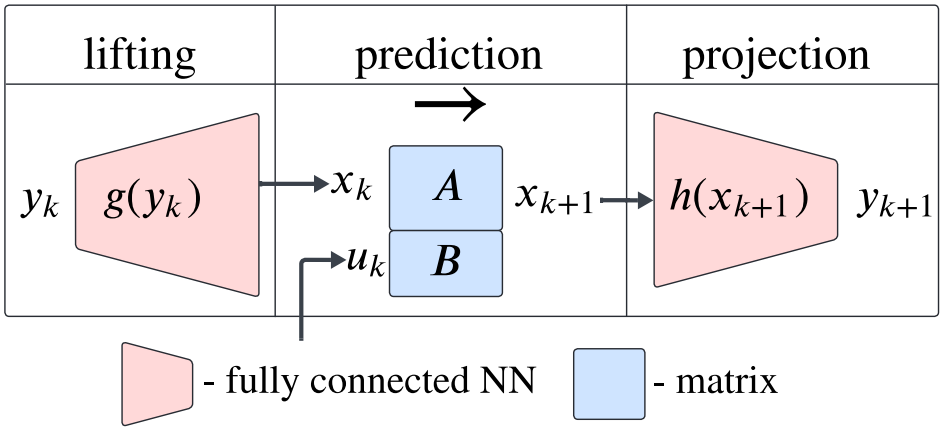


Figure 1: Architecture of the deep Koopman operator for nonautonomous systems.

introduced by Lusch et al. (2018), adapted to nonautonomous systems, as shown in Fig. 1. The control inputs u enter the scheme directly, without any transformation, so that the structure required by the MPC formulation is preserved. We use two kinds of approximations of the Koopman operator. The first one is the LTI approximation, in which the projection function is linear and is given by the matrix C, as shown in Fig. 2. In the second one, only the lifted state evolves linearly, as already shown in Fig. 1, while its projection remains nonlinear.

3. Offset-free Koopman MPC

In this section, we present the standard offset-free MPC framework that uses the LTI approximation of the Koopman operator as the prediction model. Since the observer is a standard Kalman filter, we discuss mainly the remaining two optimization problems. The LTI approximation keeps the constraints linear and the objective function quadratic, so that the standard MPC formulation applies without modification. The control structure is shown in Fig. 3.TODO: Spomenut ze toto je baseline as well as state of the art v koopmanovi?

The target optimization problem computes the steady-state targets $\bar{u}, \bar{z}, \bar{y}$ that minimize the difference between the reference y_{ref} and the reachable steady-state output $\bar{y}$. The problem is formulated as:

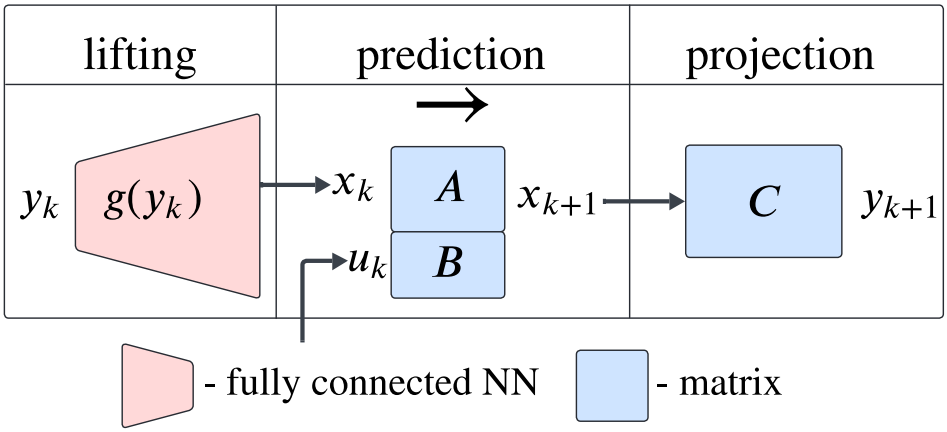


Figure 2: Architecture of the deep Koopman operator for a nonautonomous system with the linear approximation of the output map. This approximation is required for the MPC problem to remain a quadratic program.

$$\min_{\bar{u},\bar{z},\bar{y}} \quad (\bar{y} - y_{\text{ref}})^\intercal Q_{\text{y}}^{\text{t}} (\bar{y} - y_{\text{ref}}), \tag{6a}$$

$$\text{s.t.} \quad \bar{z} = A\bar{z} + B\bar{u}, \tag{6b}$$

$$\bar{y} = C\bar{z} + d, \tag{6c}$$

$$u_{\min} \leq \bar{u} \leq u_{\max}, \tag{6d}$$

$$y_{\min} \leq \bar{y} \leq y_{\max}, \tag{6e}$$

$$d = \hat{d}(t), \quad y_{\text{ref}} = y_{\text{sp}}(t), \tag{6f}$$

where $\bar{u} \in \mathbb{R}^{n_u}$, $\bar{z} \in \mathbb{R}^{n_z}$, and $\bar{y} \in \mathbb{R}^{n_y}$ are the steady-state input, lifted state, and output, respectively; $Q_{\text{y}}^{\text{t}} \succ 0$ is the output weight of the target optimization, which is tuned separately from the weights of the MPC problem; $A \in \mathbb{R}^{n_z \times n_z}$, $B \in \mathbb{R}^{n_z \times n_u}$, $C \in \mathbb{R}^{n_y \times n_z}$ are the state-space matrices of the LTI Koopman model; $d = \hat{d}(t) \in \mathbb{R}^{n_y}$ is the additive output disturbance estimate provided by the Kalman filter; $y_{\text{ref}} = y_{\text{sp}}(t)$ is the output setpoint; and $u_{\min}, u_{\max}, y_{\min}, y_{\max}$ are the input and output constraint bounds.

Given the steady-state target $(\bar{u}, \bar{z}, \bar{y})$, the MPC problem is solved at each sampling instant to compute the optimal input sequence over the horizon N:

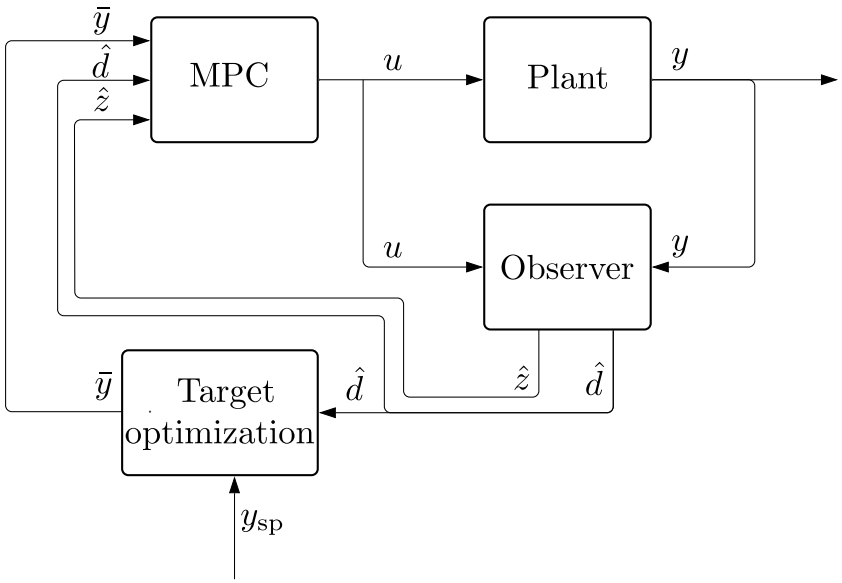


Figure 3: Basic closed-loop control structure of the offset-free MPC. The observer receives the measured output y and the applied input u and delivers the lifted state estimate $\hat{z}$ and the disturbance estimate $\hat{d}$. The target optimization receives $\hat{d}$ together with the setpoint y_{sp} and delivers the steady-state target $\bar{y}$, and the MPC block receives $\bar{y}$, $\hat{z}$, and $\hat{d}$ and delivers the input u applied to the plant.

$$\min_{u_0,\dots,u_{N-1}} \quad \sum_{k=0}^{N-1} \left[(y_k-\bar{y})^\intercal Q_\mathrm{y}(y_k-\bar{y}) + \Delta u_k^\intercal Q_{\Delta\mathrm{u}} \Delta u_k + (u_k-\bar{u})^\intercal Q_\mathrm{u}(u_k-\bar{u})\right], \tag{7a}$$

$$\text{s.t.} \quad z_{k+1} = Az_k + Bu_k, \quad k \in \mathbb{N}_0^{N-1}, \tag{7b}$$

$$y_k = Cz_k + d, \quad k \in \mathbb{N}_0^{N-1}, \tag{7c}$$

$$\Delta u_k = u_k - u_{k-1}, \quad k \in \mathbb{N}_0^{N-1}, \tag{7d}$$

$$u_\mathrm{min} \le u_k \le u_\mathrm{max}, \quad k \in \mathbb{N}_0^{N-1}, \tag{7e}$$

$$y_\mathrm{min} \le y_k \le y_\mathrm{max}, \quad k \in \mathbb{N}_0^{N-1}, \tag{7f}$$

$$z_0 = \hat{z}(t), \quad d = \hat{d}(t), \quad u_{-1} = u(t-T_\mathrm{s}), \tag{7g}$$

where N is the prediction horizon, $k \in \mathbb{N}_0^{N-1} = \{0, 1, \dots, N-1\}$ is the prediction step index, and T_s is the sampling period; $Q_\mathrm{y} \succ 0$ is the output tracking weight matrix, $Q_{\Delta\mathrm{u}} \succ 0$ the input increment weight matrix, and $Q_\mathrm{u} \succeq 0$ the weight on the deviation of the input from its steady-state target; Δu_k is the input increment at the step k; $\hat{z}(t) \in \mathbb{R}^{n_z}$ is the lifted state estimate provided by the Kalman filter; and $u_{-1} = u(t-T_\mathrm{s})$ is the input applied at the previous sampling instant, used to compute the first increment Δu_0. The first term of the objective function (7a) penalizes the predicted tracking error with respect to the output target, the second term penalizes the movement of the manipulated variables, and the third term keeps the inputs at the steady-state target computed by (6), which resolves the remaining degrees of freedom if the plant has more inputs than outputs. Following the receding-horizon principle, only the first element u_0^* of the optimal sequence is applied to the plant.

4. Methodology

In this section, we present the proposed offset-free Koopman MPC framework TODO: Hybrid KMPC? . The main drawback of the current Koopman MPC formulations, whether offset-free, robust, or of any other kind, is that the optimization is carried out without fully exploiting the knowledge of the lifted space. The prediction is made in the lifted space, but the objective function is expressed in the original space, which is recovered from the lifted state by the linear transformation $y_k = Cz_k$. This recovery is nonlinear by

nature, $y_k = h(z_k)$, and it is restricted to a linear map only so that the MPC problem remains a quadratic program. Formulations that do not use this transformation directly usually rely on an equivalent one, for example by augmenting the lifted state with the measurements, in which case the recovery of the measurements from the lifted state is again linear. TODO: Intro w citation?

We show that the lifted space can be used directly in the objective function, and we derive the corresponding transformation of the weighting matrix. The back transformation is thereby eliminated from the objective function of the MPC problem, while it is still required to compute the steady-state target and to impose the output constraints. In both cases, we approximate it by a first-order Taylor expansion.

The proposed approach applies to any Koopman model whose lifting and projection are nonlinear, independently of the concrete identification. Either the lifting and the projection are given by one invertible function, as in eDMD, or they are two distinct functions represented by neural networks, as in this work.

4.1. Linearization of the output mapping

For the control problem, the transformation from the lifted space back to the original space cannot be eliminated completely, because both the steady-state target and the output constraints are expressed in the original variables. We therefore approximate the output mapping by a first-order Taylor expansion, and we consider two linearization points, the target computed at the previous sampling instant and the current state estimate, which gives

$$y_k = h(z_k) \approx h(\bar{z}(t - T_\mathrm{s})) + H(\bar{z}(t - T_\mathrm{s}))(z_k - \bar{z}(t - T_\mathrm{s}))\,, \tag{8a}$$

$$y_k = h(z_k) \approx h(\hat{z}(t)) + H(\hat{z}(t))(z_k - \hat{z}(t))\,, \tag{8b}$$

where $h : \mathbb{R}^{n_z} \to \mathbb{R}^{n_y}$ is the decoder of the identified model, which acts as the nonlinear output mapping; $H(\cdot) \in \mathbb{R}^{n_y \times n_z}$ is the Jacobian of h evaluated at its argument; and $\bar{z}(t - T_\mathrm{s})$ is the steady-state lifted state computed by the target optimizer at the previous sampling instant.

Both expansions are affine in the lifted state, and they differ only in the point at which the decoder and its Jacobian are evaluated. Denoting this point by z_LP and collecting the two terms that do not depend on z_k into a single offset, the output equation reads

$$y_k = H(z_\mathrm{LP})\, z_k + c(t), \qquad c(t) = h(z_\mathrm{LP}) - H(z_\mathrm{LP})\, z_\mathrm{LP}, \tag{9}$$

where z_{LP} is the linearization point, equal to $\bar{z}(t - T_{\mathrm{s}})$ in (8a) and to $\hat{z}(t)$ in (8b), and $c(t) \in \mathbb{R}^{n_y}$ is the linearization offset. The Jacobian $H(z_{\mathrm{LP}})$ and the offset $c(t)$ are evaluated once per sampling instant and are therefore constant over the whole prediction horizon of dynamic optimization, while the lifted state z_k is the only quantity that varies along it. The output equation thus keeps the structure of the LTI model of Sec. 3, with the constant output matrix C replaced by the Jacobian at the linearization point and with the offset $c(t)$ added, so that every optimization problem in which (9) is substituted remains a quadratic program.

The affine form is accurate only in a neighborhood of z_{LP}, and each of the two candidate points of (8) favors one of the two optimization problems. The target optimization evaluates the output equation at the steady-state lifted state $\bar{z}$, which the target $\bar{z}(t - T_{\mathrm{s}})$ of the previous sampling instant approximates closely, whereas the dynamic optimization evaluates it along a predicted trajectory that starts at the current estimate $\hat{z}(t)$, so that this second point favors the early part of the prediction horizon. The two problems nevertheless have to share a single linearization point, because the objective function of the dynamic optimization tracks the target delivered by the target optimization, and a target computed under one approximation of the output mapping would not be consistent with predictions made under another. Once the plant settles at the requested operating point, the estimate converges to the target and both candidate points coincide with it, so that the two choices are equivalent in the steady state and differ only during the transients. For this reason, both of them are implemented and compared in this work, in the two closed loops summarized in Algorithm 1 and Algorithm 2, and their behavior is examined in Sec. 7 and Sec. 8.

4.2. Target optimization

The objective function of the target optimization is the same as in the standard offset-free MPC of Sec. 3. The difference is in the output equation, where the linear map of (6c) is replaced by the affine form (9) of the nonlinear output mapping. The proposed target optimization problem is therefore formulated as:

$$
\begin{aligned}
\min_{\bar{u},\bar{z},\bar{y}} \quad & (\bar{y} - y_{\mathrm{ref}})^{\intercal} Q_{\mathrm{y}}^{\mathrm{t}} (\bar{y} - y_{\mathrm{ref}}), && \text{(10a)}\\
\text{s.t.} \quad & \bar{z} = A\bar{z} + B\bar{u}, && \text{(10b)}\\
& \bar{y} = H(z_{\mathrm{LP}})\bar{z} + h(z_{\mathrm{LP}}) - H(z_{\mathrm{LP}})z_{\mathrm{LP}} + d, && \text{(10c)}\\
& u_{\min} \leq \bar{u} \leq u_{\max}, && \text{(10d)}\\
& y_{\min} \leq \bar{y} \leq y_{\max}, && \text{(10e)}\\
& d = \hat{d}(t), \quad y_{\mathrm{ref}} = y_{\mathrm{sp}}(t), && \text{(10f)}
\end{aligned}
$$

where the output equation (10c) is the affine form (9) written out at the linearization point z_{LP}, which (10f) fixes to either of the two candidates of (8). The problem is stated for both of them, since the formulation does not depend on which one is taken, and the same point is then used by the dynamic optimization of Sec. 4.3. The two candidates differ in the instant at which the point becomes available. The target $\bar{z}(t - T_{\mathrm{s}})$ is delivered by this problem at the previous sampling instant and is therefore known before the measurement of the current one is processed, whereas the state estimate $\hat{z}(t)$ is delivered by the observer of the current sampling instant and is therefore available only after the measurement update. This difference fixes the order in which the blocks of the closed loop are evaluated, as described in Sec. 4.4.

4.3. Dynamic optimization

The dynamic optimization differs from the standard formulation (7) in the objective function and in the output equation, while the prediction model, the constraints on the inputs, and the input increments are kept unchanged.

The proposed MPC problem is formulated as:

$$\min_{u_0,\ldots,u_{N-1}} \quad \sum_{k=0}^{N-1} \left[(z_k - \bar{z})^\intercal Q_\mathrm{z}(z_k - \bar{z}) + \Delta u_k^\intercal Q_{\Delta\mathrm{u}} \Delta u_k + (u_k - \bar{u})^\intercal Q_\mathrm{u}(u_k - \bar{u})\right], \tag{11a}$$

$$\text{s.t.} \quad z_{k+1} = Az_k + Bu_k, \quad k \in \mathbb{N}_0^{N-1}, \tag{11b}$$

$$y_k = H(z_\mathrm{LP})z_k + h(z_\mathrm{LP}) - H(z_\mathrm{LP})z_\mathrm{LP} + d, \quad k \in \mathbb{N}_0^{N-1}, \tag{11c}$$

$$\Delta u_k = u_k - u_{k-1}, \quad k \in \mathbb{N}_0^{N-1}, \tag{11d}$$

$$u_\mathrm{min} \leq u_k \leq u_\mathrm{max}, \quad k \in \mathbb{N}_0^{N-1}, \tag{11e}$$

$$y_\mathrm{min} \leq y_k \leq y_\mathrm{max}, \quad k \in \mathbb{N}_0^{N-1}, \tag{11f}$$

$$z_0 = \hat{z}(t), \quad d = \hat{d}(t), \quad u_{-1} = u(t - T_\mathrm{s}), \tag{11g}$$

where the linearization point z_LP is the same as in the target optimization problem (10), so that a single linearization of the output mapping is shared by both problems at every sampling instant. The first-order Taylor expansion provides y_k, which enters the problem only through the output constraints (11f).

In the objective function (11a), the deviation from the target is penalized directly in the lifted space by the weighting matrix Q_z, which we propose to obtain from the output tracking term of the standard formulation (7a) by substituting the affine form (9) for both the predicted output y_k and the output target $\bar{y}$. The two are evaluated at the same linearization point and carry the same disturbance estimate, so that the offset $c(t)$ and the disturbance d cancel in their difference and the tracking term reduces to a quadratic form in the lifted deviation, which gives

$$\begin{aligned} y_k - \bar{y} &= \big(H(z_\mathrm{LP})z_k + c(t) + d\big) - \big(H(z_\mathrm{LP})\bar{z} + c(t) + d\big) \\ &= H(z_\mathrm{LP})\big(z_k - \bar{z}\big), \end{aligned} \tag{12a}$$

$$(y_k - \bar{y})^\intercal Q_\mathrm{y}(y_k - \bar{y}) = (z_k - \bar{z})^\intercal Q_\mathrm{z}(z_k - \bar{z}), \tag{12b}$$

$$Q_\mathrm{z} = H(z_\mathrm{LP})^\intercal Q_\mathrm{y} H(z_\mathrm{LP}), \tag{12c}$$

where the Jacobian is absorbed into the weighting matrix (12c), which leaves the tracking term (12b) entirely in the lifted coordinates and is the matrix that enters the objective function (11a). The derivation holds for either of the two linearization points, since both the prediction and the target are expanded at whichever one is selected.

In this way, the controller is still tuned through the matrix Q_{y}, which acts on the physical outputs and is therefore far more convenient to choose than a weighting matrix acting directly on the lifted states, whose entries carry no physical meaning. The tuning of the proposed formulation is kept equivalent to the tuning of the standard formulation (7). The matrix Q_{z} constructed by (12c) is positive semidefinite with rank at most n_y.

This is a substantial departure from the standard Koopman MPC, in which the objective function is defined in the original space. If no output constraints are imposed, the predicted output y_k disappears from the problem altogether, since it enters only through (11f), and the dynamic optimization is then solved entirely in the lifted space without any approximation of the output mapping.

4.4. Closed-loop control structure

The closed-loop structure follows the one shown in Fig. 3, with three modifications: (i) a linearization block is added, (ii) the observer is a time-varying Kalman filter, and (iii) the two optimization problems are replaced by (10) and (11). The order in which the blocks are evaluated within one sampling period depends on the chosen linearization point. Algorithm 1 summarizes the loop for the linearization at the previous target.

Algorithm 1 Closed-loop control with nonlinear output mapping and linearization at the previous target

Require: Koopman lifting function $g(\cdot)$, initial measurement $y(0)$

1: **Initialization:** $\hat{z}(0) \leftarrow g(y(0))$, $\hat{d}(0) \leftarrow 0$; solve (10) with $z_{\mathrm{LP}} = \hat{z}(0)$ and $\hat{d}(0)$ to obtain $\bar{z}(-T_{\mathrm{s}})$
2: **loop**
3: **Measurement:** Obtain $y(t)$ from the plant
4: **Linearization:** Set $z_{\mathrm{LP}} \leftarrow \bar{z}(t - T_{\mathrm{s}})$ and compute $H(z_{\mathrm{LP}})$ and $h(z_{\mathrm{LP}})$
5: **Observer:** Run the time-varying Kalman filter with $y(t)$, $u(t - T_{\mathrm{s}})$ and $H(z_{\mathrm{LP}})$ to obtain the estimates $\hat{z}(t)$ and $\hat{d}(t)$
6: **Target estimation:** Solve (10) with $\hat{d}(t)$ to obtain $\bar{z}(t)$, $\bar{u}(t)$, $\bar{y}(t)$
7: **MPC:** Solve (11) with $\hat{z}(t)$, $\hat{d}(t)$, $\bar{z}(t)$ to obtain the optimal first input u_0^*
8: **Apply input:** Apply $u(t) = u_0^*$ to the plant
9: **end loop**

In Algorithm 1, the encoder g provides the initial lifted-state estimate from the first measurement, and the disturbance estimate is initialized at zero. Because the linearization point is the target of the previous sampling instant, the output mapping is linearized before the observer is run, and the same Jacobian is then used by the observer, by the target optimizer, and by the MPC problem. Algorithm 2 summarizes the loop for the linearization at the current state estimate.

Algorithm 2 Closed-loop control with nonlinear output mapping and linearization at the current state estimate

Require: Koopman lifting function $g(\cdot)$, initial measurement $y(0)$

1: **Initialization:** $\hat{z}(0) \leftarrow g(y(0))$, $\hat{d}(0) \leftarrow 0$
2: **loop**
3: **Measurement:** Obtain $y(t)$ from the plant
4: **Observer:** Run the time-varying Kalman filter with $y(t)$, $u(t - T_\mathrm{s})$ and $H(\hat{z}(t - T_\mathrm{s}))$ to obtain the estimates $\hat{z}(t)$ and $\hat{d}(t)$
5: **Linearization:** Set $z_\mathrm{LP} \leftarrow \hat{z}(t)$ and compute $H(z_\mathrm{LP})$ and $h(z_\mathrm{LP})$
6: **Target estimation:** Solve (10) with $\hat{d}(t)$ to obtain $\bar{z}(t)$, $\bar{u}(t)$, $\bar{y}(t)$
7: **MPC:** Solve (11) with $\hat{z}(t)$, $\hat{d}(t)$, $\bar{z}(t)$ to obtain the optimal first input u_0^*
8: **Apply input:** Apply $u(t) = u_0^*$ to the plant
9: **end loop**

The two algorithms differ in the linearization point and, as a consequence, in the position of the linearization step within the sampling period. In Algorithm 1, a single Jacobian is evaluated at the target of the previous sampling instant and is shared by the observer, by the target optimizer, and by the MPC problem. In Algorithm 2, the target optimizer and the MPC problem use the Jacobian at the current state estimate, which becomes available only after the measurement update. The observer itself cannot use this Jacobian, since it is the quantity that the update produces, and its measurement equation is therefore linearized at the estimate of the previous sampling instant.

4.5. State estimation with a time-varying Kalman filter

The observer estimates the augmented state $\tilde{z}(t) = [z(t)^\intercal,\ d(t)^\intercal]^\intercal$ from the noisy output $y(t)$ and the applied input $u(t)$. The augmented linear dynamics are identical to (1) with the original state replaced by the lifted

Koopman state, resulting in

$$\tilde{z}(t+T_\mathrm{s}) = \tilde{A}\,\tilde{z}(t) + \tilde{B}\,u(t), \qquad \tilde{A} = \begin{bmatrix} A & B_d \\ 0 & I \end{bmatrix}, \quad \tilde{B} = \begin{bmatrix} B \\ 0 \end{bmatrix}, \tag{13}$$

where $\tilde{z}(t) \in \mathbb{R}^{n_z+n_d}$ is the augmented state stacking the lifted Koopman state and the output disturbance, and $\tilde{A} \in \mathbb{R}^{(n_z+n_d)\times(n_z+n_d)}$ and $\tilde{B} \in \mathbb{R}^{(n_z+n_d)\times n_u}$ are the augmented system matrices.

Because the output mapping is nonlinear, the measurement equation is relinearized at each sampling instant around the linearization point z_LP selected by Algorithm 1 or Algorithm 2, which gives

$$y(t) \approx \tilde{C}(t)\,\tilde{z}(t) + c(t), \qquad \tilde{C}(t) = \begin{bmatrix} H(z_\mathrm{LP}),\ C_d \end{bmatrix}, \quad c(t) = h(z_\mathrm{LP}) - H(z_\mathrm{LP})\,z_\mathrm{LP}, \tag{14}$$

where $\tilde{C}(t) \in \mathbb{R}^{n_y\times(n_z+n_d)}$ is the time-varying output matrix formed by stacking the Jacobian of h at the linearization point and the disturbance output matrix C_d, and $c(t)$ is the offset of (9).

With the linear prediction model (13) and the time-varying output matrix (14), the filter runs the standard predict–update recursion at each sampling instant. The prediction step reads

$$\hat{\tilde{z}}^{-}(t) = \tilde{A}\,\hat{\tilde{z}}(t-T_\mathrm{s}) + \tilde{B}\,u(t-T_\mathrm{s}), \tag{15a}$$

$$P^{-}(t) = \tilde{A}\,P(t-T_\mathrm{s})\,\tilde{A}^{\intercal} + Q, \tag{15b}$$

where $\hat{\tilde{z}}^{-}(t) \in \mathbb{R}^{n_z+n_d}$ is the prior augmented-state estimate, $P^{-}(t) \in \mathbb{R}^{(n_z+n_d)\times(n_z+n_d)}$ is the prior error covariance, and $Q \succeq 0$ is the process noise covariance matrix. The superscript minus sign distinguishes the quantities available before the measurement $y(t)$ has been processed. The update step is then given by

$$S(t) = \tilde{C}(t)\,P^{-}(t)\,\tilde{C}(t)^{\intercal} + R, \tag{16a}$$

$$K(t) = P^{-}(t)\,\tilde{C}(t)^{\intercal}\,S(t)^{-1}, \tag{16b}$$

$$\hat{\tilde{z}}(t) = \hat{\tilde{z}}^{-}(t) + K(t)\Big(y(t) - \tilde{C}(t)\,\hat{\tilde{z}}^{-}(t) - c(t)\Big), \tag{16c}$$

$$P(t) = \Big(I - K(t)\,\tilde{C}(t)\Big)\,P^{-}(t), \tag{16d}$$

where $S(t) \in \mathbb{R}^{n_y\times n_y}$ is the innovation covariance, $K(t) \in \mathbb{R}^{(n_z+n_d)\times n_y}$ is the Kalman gain, $\hat{\tilde{z}}(t)$ is the posterior augmented-state estimate whose upper n_z entries give $\hat{z}(t)$ and lower n_d entries give $\hat{d}(t)$, $P(t)$ is the posterior error covariance, and $R \succ 0$ is the measurement noise covariance matrix.

4.6. Block-diagonal deep Koopman transformation

In this work, we encountered suboptimal or infeasible solutions of the Koopman MPC problem when it was solved with the Gurobi solver. To improve the behavior of the optimization, we propose to bring the matrix A into a block-diagonal structure. The transformation is applied after the Koopman operator has been identified, and it amounts to finding a transformation matrix V such that

$$A^{\mathrm{bd}} = V^{-1} A V, \tag{17}$$

where $V \in \mathbb{R}^{n_z \times n_z}$ is constructed from the eigenvectors of A and A^{bd} is the real block-diagonal form of A, in which the real eigenvalues form one-by-one blocks and the complex conjugate pairs form two-by-two blocks. The remaining parts of the model have to be modified accordingly, and the same matrix V is used to transform the input and output matrices as well as the lifted state, which gives

$$B^{\mathrm{bd}} = V^{-1} B, \quad C^{\mathrm{bd}} = C V, \quad z_k^{\mathrm{bd}} = V^{-1} z_k. \tag{18}$$

The Jacobian of the output mapping is transformed in the same way, resulting in

$$H^{\mathrm{bd}}(z_k) = H(z_k) V = H(V z_k^{\mathrm{bd}}) V. \tag{19}$$

For a fair comparison, this transformation is applied to every model identified in this work, not only to the Koopman models.

5. Identification

In this section, we introduce the benchmark process, the data collected from it, and the three prediction models whose closed-loop performance is compared in Sec. 7.

5.1. Reactor–reactor–separator process

The considered plant consists of two continuous stirred tank reactors in series followed by a flash separator, with a recycle stream from the separator back to the first reactor. The plant structure follows Liu et al. (2009), and the kinetic, thermodynamic, and geometric parameters are those reported by Li and Swartz (2019). Two consecutive first-order reactions A $\rightarrow$ B $\rightarrow$ C take place in both reactors, where B is the desired product and C an undesired by-product. The control of the plant is therefore a selectivity problem, since

additional heat increases the conversion of the reactant A but also the loss of the product B into C.

The plant has nine states, namely the mass fractions of A and B and the temperature in each of the three vessels, and the third vessel is the separator, in which no reaction occurs. The manipulated variables are the three heat duties Q_1, Q_2, Q_3 and the fresh feed flow rate F_{10} to the first reactor. The fresh feed flow rate F_{20} to the second reactor and the recycle flow rate F_{r} are held fixed at the nominal values $0.5\,\mathrm{m}^3/\mathrm{s}$ and $4.0\,\mathrm{m}^3/\mathrm{s}$. The measured and controlled outputs are the three vessel temperatures T_1, T_2, T_3 and the bottoms quality x_{B3}, which is the mass fraction of B leaving the separator. The vessel inventories are constant, so the internal flow rates follow from F_{10} together with two fixed flows. The nominal operating point and the box within which the manipulated inputs are moved during identification are listed in Table 1.

Table 1: Manipulated variables of the reactor–separator plant, with the nominal operating point of Li and Swartz (2019) and the box used for the identification experiment.

Variable	Description	Nominal	Range
Q_1, Q_2, Q_3 $[\mathrm{kJ\,s^{-1}}]$	Heat duties of the three vessels	10.0	[0.0, 25.0]
F_{10} $[\mathrm{m^3\,s^{-1}}]$	Fresh feed to the first reactor	8.3	[3.0, 16.0]

Two mechanisms make the plant nonlinear. The first one is the Arrhenius dependence of both reaction rates on the vessel temperatures, and the second one is the relative-volatility flash of the separator, which makes the composition of the recycle stream a nonlinear function of the composition in the separator and closes a nonlinear loop back to the first reactor. As a consequence, the bottoms quality is not monotone in the supplied heat: the yield of B reaches a peak and then decreases again as more of the product is converted into C. A model identified around a single operating point can therefore reproduce only one branch of this behavior, which is the reason for choosing this plant as the benchmark. The plant equations are integrated in Python with the explicit Runge–Kutta solver of SciPy, and the sampling period used for identification and for control is $T_{\mathrm{s}} = 1\,\mathrm{s}$.

5.2. Identification data

The excitation is a sequence of 100 piecewise-constant input steps, each held for 180 s, which gives 18 000 samples of input–output data. The hold

time of 180 s is long enough for the plant to settle at a new steady state after every step, so that the data contain both the transient and the steady-state response to each input change. All four manipulated inputs are stepped within the box of Table 1. Over the whole data set the temperatures cover the range from 326 K to 396 K and the bottoms quality the range from 0.40 to 0.76.

Zero-mean Gaussian measurement noise is added to all four outputs, with a standard deviation equal to 1% of the range of the respective output, that is 0.487 K, 0.527 K, and 0.610 K for the three temperatures and 0.0036 for the bottoms quality. The data set is split into the first 12 960 (72%) samples used for training, the following 2880 (16%) samples used for validation and early stopping, and the remaining 2160 (12%) samples, which are never seen during identification and serve as the test data. Inputs and outputs are standardized to zero mean and unit variance using the statistics of the training split only, and all three models are identified in these coordinates, so that the different scales of the temperatures and of the mass fraction do not bias the identification.

5.3. Subspace model

The industrial baseline model is identified with numerical algorithms for subspace state-space system identification (N4SID) Van Overschee and De Moor (2012), as implemented in the SIPPY package Armenise et al. (2018). The direct input–output term is fixed to zero, and the past and future horizons are both held at 20 samples. The model of order 6 attains the smallest mean absolute error (MAE) on the standardized validation split, and this model, whose spectral radius is 0.9865, is used for the closed-loop comparison.

5.4. Koopman models

The two Koopman models share the structure of the encoder and of the linear lifted dynamics, but each model is trained separately with its own hyperparameters, and they differ in the map from the lifted state to the output. The first model restricts this map to a matrix C, so the output is a linear projection of the lifted state. It is the LTI model required by the standard offset-free Koopman MPC of Sec. 3, and it is the model used by the Koopman baseline in Sec. 7.

The second model keeps the nonlinear output map $h(\cdot)$ as proposed in Sec. 4, realized by a decoder whose hidden layer mirrors the encoder. This

model is used by both variants of the proposed formulation, Algorithm 1 and Algorithm 2. The two variants share the network, the weights, and the block-diagonal transformation of Sec. 4.6, and they differ only in the point at which $h(\cdot)$ is linearized. Any difference between them in Sec. 7 is therefore attributable to that linearization point.

The hyperparameters of each Koopman model are selected by a separate search in Optuna Akiba et al. (2019), and both networks are trained with the Adam optimizer in PyTorch Paszke et al. (2019) through the Neuromancer library Drgoňa et al. (2023). The selected values are listed in Table 2, where w_{y}, w_{z}, and w_{r} are the weights of the output, latent-state, and reconstruction terms of the training loss, and N_{r} is the length of the training window, as in the Lusch et al. (2018).

Table 2: Selected hyperparameters of the Koopman models. The columns are the model with the linear decoder and the model with the nonlinear decoder. The number of trainable parameters and the spectral radius are properties of the model selected by the search.

	Linear decoder	Nonlinear decoder
Lifted dimension n_z	26	24
Encoder hidden layers	1	1
Hidden-layer width	20	5
Activation	ELU	ELU
Training window N_{r}	20	120
Minibatch size	40	40
Epoch budget	500	4000
Learning rate	$1.59 \cdot 10^{-3}$	$1.36 \cdot 10^{-3}$
Output weight w_{y}	1.77	5.99
Latent weight w_{z}	10.01	1.48
Reconstruction weight w_{r}	4.85	1.29
Trainable parameters	1530	990
Spectral radius	0.9854	0.9499

5.5. Open-loop accuracy on the test data

To compare the models on the data that none of them has seen, each model is initialized from the first sample of the test split and then run in open loop over the whole test window, driven only by the recorded inputs and without any state correction. The prediction errors are evaluated against

the noise-free outputs of the plant, which are available because the plant is simulated, so that the reported numbers reflect the model error alone and are not inflated by the measurement noise. Fig. 4 shows the first four input steps of this window and Table 3 summarizes the accuracy over the whole window.

Table 3: Open-loop prediction accuracy on the test data, measured by the MAE against the noise-free plant outputs. The last column is the sum of the MAEs of the four standardized outputs, which is the criterion on which the models are directly comparable.

Model	T_1 [K]	T_2 [K]	T_3 [K]	x_{B3} [–]	Σ MAE [–]
N4SID, order 6	3.53	3.43	3.83	0.0239	1.18
Koopman, linear h, $n_z = 26$	3.67	3.56	4.05	0.0229	1.21
Koopman, nonlinear h, $n_z = 24$	0.56	0.66	0.74	0.0040	0.21

The Koopman model with the nonlinear decoder is significantly more accurate than both linear alternatives, and it reduces the sum of the standardized MAEs by approximately 82% with respect to the subspace model. Its prediction error is of the same magnitude as the standard deviation of the measurement noise on all four outputs. In contrast, the Koopman model with the linear decoder is not better than the subspace model in open loop, and its summed standardized error is approximately 2% higher, even though its lifted dimension exceeds the order of the subspace model. Restricting the decoder to a matrix leaves an input–output map of an LTI system, for which subspace identification is a mature and well-tuned tool. This model is nevertheless retained in the comparison, because it is the model that the standard offset-free Koopman MPC formulation of Sec. 3 requires, and that is widely used in Koopman MPC papers, it therefore isolates the effect of the nonlinear output mapping of Sec. 4 from the effect of the lifting itself. Note that a lower open-loop prediction error does not by itself guarantee a better closed-loop performance, which is therefore assessed separately in Sec. 7.

6. Control setup

In this section, we describe the closed-loop scenario on which the models of Sec. 5 are compared, together with [claude-opus-5.5] **the tuning and the observers of the compared controllers**[/claude-opus-5.5]. The scenario, the weights, the constrainxts, and the disturbance sequence are identical in all

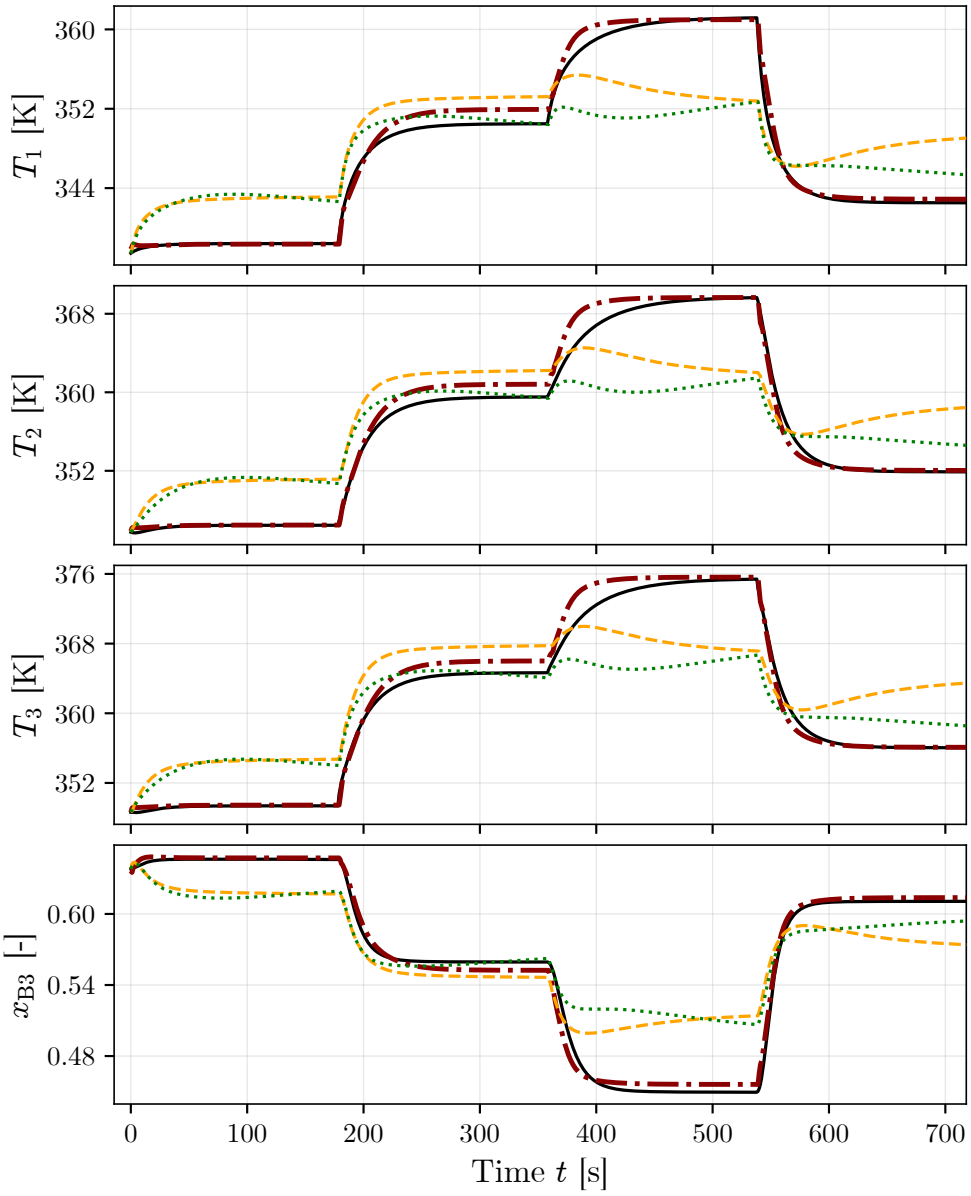


Figure 4: Open-loop predictions of the identified models on the first four input steps of the test data. The **black** solid line is the noise-free plant, the dash-dotted red line is the Koopman model with the nonlinear decoder, the dashed orange line is the Koopman model with the linear decoder, and the dotted green line is the N4SID model.

runs reported in Sec. 7, so that the differences between the controllers are attributable to the prediction model and to the way the output mapping enters the control problem, and not to the way the loop was set up.

All four measured outputs T_1, T_2, T_3, and x_{B3} are controlled, and the four manipulated variables of Table 1 are available to the controller, so the plant is square and the output specifications determine the steady-state target. The sampling period is the one used for identification, $T_s = 1\,\mathrm{s}$, and the prediction horizon is $N = 60$ samples. The input constraints (11e) are the identification box of Table 1. The output constraints (11f) are an operating window rather than equipment limits, namely $310\,\mathrm{K} \leq T_i \leq 380\,\mathrm{K}$ for the three vessel temperatures and $0.35 \leq x_{B3} \leq 0.85$ for the bottoms qualityTODO: actually neccessary? not hitting, also make table not in text? . All signals, weights, and bounds are expressed in the standardized coordinates of Sec. 5, so that the three temperatures and the mass fraction contribute to the objective function on a comparable scale.

6.1. Simulation scenario and disturbances

The scenario is a grade campaign on the bottoms quality, combined with two unmeasured feed disturbances, and its schedule is listed in Table 4. The plant starts at the nominal steady state of Li and Swartz (2019). The campaign then switches to a premium grade with a higher bottoms quality and later returns to the nominal grade. Both grades are steady states of the plant reachable inside the identification box, and the setpoints of the three temperatures are the corresponding steady-state values, so the controller has to move the whole operating point and not the quality alone. While the premium grade is held, the fresh feed first becomes colder, which represents a drift of the feed preheater, and then leaner in the reactant A, which represents a switch of the feed tank, and both disturbances persist until the end of the simulation.

Both disturbances act on the plant only, and the encoder, the observer, the target optimizer, and the MPC problem are left unchanged, so that the four controllers built on an identified model can reject them only through the output disturbance estimate $\hat{d}(t)$. Their magnitudes are deliberately modest, so that both grades remain reachable inside the identification box after the disturbances. The measurements are noise-free in all closed-loop runs.

Table 4: Schedule of the closed-loop campaign. Each row lists the output setpoints and the fresh-feed conditions in force from the given time onwards, and the simulation ends at $t = 1000\,\mathrm{s}$. The feed conditions act on the plant only and are not measured by any controller.

		Setpoint				Fresh feed	
t [s]	Event	T_1 [K]	T_2 [K]	T_3 [K]	x_{B3} [–]	Temperature [K]	Fraction of A [–]
0	Nominal grade	336.7	344.1	346.2	0.637	313	1.00
100	Premium grade	331.5	338.6	340.1	0.702	313	1.00
350	Colder feed	331.5	338.6	340.1	0.702	311	1.00
600	Leaner feed	331.5	338.6	340.1	0.702	311	0.98
750	Nominal grade	336.7	344.1	346.2	0.637	311	0.98

6.2. Tuning

The target optimization (10) uses $Q_{\mathrm{y}}^{\mathrm{t}} = \mathrm{diag}([1,\,1,\,1,\,20])$, so that the steady-state target is the reachable operating point closest to the requested grade, with the bottoms quality weighted an order of magnitude above the temperatures. The MPC problem (11) is tuned in the original setup by

$$\begin{gathered} Q_{\mathrm{y}} = \mathrm{diag}([2,\,2,\,2,\,15]), \quad Q_{\Delta\mathrm{u}} = \mathrm{diag}([0.5,\,0.5,\,0.5,\,1]), \\ Q_{\mathrm{u}} = \mathrm{diag}([0.2,\,0.2,\,0.2,\,0.5]), \end{gathered} \tag{20}$$

where the entries of Q_{y} follow the order T_1, T_2, T_3, x_{B3} of the outputs and the entries of $Q_{\Delta\mathrm{u}}$ and Q_{u} the order Q_1, Q_2, Q_3, F_{10} of the inputs. The bottoms quality again carries the largest output weight, because it is the variable that the campaign has to meet, while the temperatures are regulated but act mainly as operating limits. The input target towards which Q_{u} pulls is the nominal utility and throughput setting of Table 1, held constant over the whole campaign, so that this term expresses the economic preference of the plant rather than a tracking requirement.

6.3. Observers

Each controller is paired with the observer that its formulation requires. The two controllers built on an LTI model, namely the one using the subspace model and the one using the Koopman model with the linear decoder, use a stationary Kalman filter on the augmented system (1), while the proposed controllers use the time-varying filter of Sec. 4.5. The number of estimated

disturbances equals the number of measurements, $n_d = n_y = 4$, with $B_d = 0$ and $C_d = I$, so that the disturbance is purely additive on the outputs and the rank condition (2) is satisfied. The covariances in the standardized coordinates are $P(0) = I$, $R = 0.3\,I$, and $Q = \mathrm{diag}([0.1\,I,\ 0.1\,I])$, with the two blocks acting on the lifted state and on the disturbance, respectively.

6.4. Evaluation

To evaluate the closed-loop performance, we accumulate over the whole simulation the same quantities that the MPC problem penalizes, which gives

$$J_{\mathrm{y}} = \sum_{t \in \mathcal{T}} \left\| y(t) - y_{\mathrm{sp}}(t) \right\|^2_{Q_{\mathrm{y}}}, \tag{21a}$$

$$J_{\Delta \mathrm{u}} = \sum_{t \in \mathcal{T}} \left\| u(t) - u(t - T_{\mathrm{s}}) \right\|^2_{Q_{\Delta \mathrm{u}}}, \tag{21b}$$

$$J_{\mathrm{u}} = \sum_{t \in \mathcal{T}} \left\| u(t) - \bar{u}(t) \right\|^2_{Q_{\mathrm{u}}}, \tag{21c}$$

$$J = J_{\mathrm{y}} + J_{\Delta \mathrm{u}} + J_{\mathrm{u}}, \tag{21d}$$

where J_{y} is the closed-loop output cost, $\|v\|^2_Q = v^\intercal Q v$, and $\mathcal{T} = \{0,\ T_{\mathrm{s}},\ \ldots,\ (n_{\mathrm{sim}} - 1)T_{\mathrm{s}}\}$ is the set of the sampling instants of the campaign, with [claude-opus-5.5]$n_{\mathrm{sim}} = 1000$[/claude-opus-5.5] its length in samples; $J_{\Delta \mathrm{u}}$ is the closed-loop input-increment cost and J_{u} the closed-loop input-target cost; and J is their sum. The three parts are the closed-loop counterparts of the three terms of (7a) and are evaluated with the weights of the tuning that is being reported.

The target optimization (10) and the MPC problem (11) are quadratic programs, formulated in CVXPY and solved with Gurobi Gurobi Optimization, LLC (2024). The decoder $h(z_{\mathrm{LP}})$ and its Jacobian $H(z_{\mathrm{LP}})$ are evaluated in PyTorch Paszke et al. (2019) at every sampling instant. All closed-loop runs were carried out on a Mac mini with an Apple M4 Pro processor. TODO: add bibliography entries for CVXPY

7. Closed-loop results

In this section, we report the closed-loop results of the simulation scenario of Sec. 6. Four controllers are compared, namely the offset-free MPC of Sec. 3 using the N4SID model, the same formulation using the Koopman

model with the linear decoder, and the proposed formulation of Sec. 4 in its two variants, linearized at the target of the previous sampling instant as in Algorithm 1 and at the current state estimate as in Algorithm 2. The N4SID controller is the reference to which the closed-loop objective is normalized, as a representation of the industrial benchmark. Fig. 5 and Fig. 6 show the controlled outputs and the manipulated variables in the original setup (20), Table 5 lists the closed-loop objective (21) and its three parts in the same setup. The interpretation of these results is given in Sec. 8.

Table 5: Closed-loop performance in the original setup (20). The three parts of the objective function are the ones defined in (21), and the last column is normalized so that 100% stands for the performance of the N4SID controller.

Controller	J_y	$J_{\Delta\mathrm{u}}$	J_u	J	J/J_N4SID [%]
N4SID, order 6	101.7	12.7	21.9	136.3	100.0
Koopman, linear h	104.6	8.5	12.6	125.8	92.2
Proposed, $z_\mathrm{LP} = \bar{z}(t - T_\mathrm{s})$	93.2	9.0	9.3	111.5	81.8
Proposed, $z_\mathrm{LP} = \hat{z}(t)$	92.9	9.1	10.2	112.2	82.3

Table 5 lists the closed-loop objective in the original setup. Compared with the industrial benchmark, represented by the MPC using the N4SID model, the proposed variants linearized at the previous target and at the current state estimate improve the closed-loop performance by 18.2% and 17.7%, respectively. Compared with the state-of-the-art deep Koopman MPC, which uses the linear decoder and reaches 7.8% improvement over the N4SID objective, the proposed variants improve the closed-loop performance by a further 10.5% and 9.9%, respectively.

To assess the sensitivity of these results to the tuning, the campaign was repeated with the output weight divided by five, $Q_\mathrm{y} = \mathrm{diag}([0.4,\ 0.4,\ 0.4,\ 3])$, and multiplied by five, $Q_\mathrm{y} = \mathrm{diag}([10,\ 10,\ 10,\ 75])$, with the remaining weights of (20) unchanged and the objective accumulated with the weights of each tuning. Fig. 7 shows the total objective of the two proposed variants relative to the linear-decoder controller at the same tuning. With Q_y divided by five, the variants linearized at the previous target and at the current state estimate reach 94.5% and 94.3%, respectively. In the original setup they reach 88.7% and 89.2%, and with Q_y multiplied by five they reach 83.2% and 85.9%. The relative objective of both variants thus decreases as the output weight increases. This is expected, as the improvement of proposed

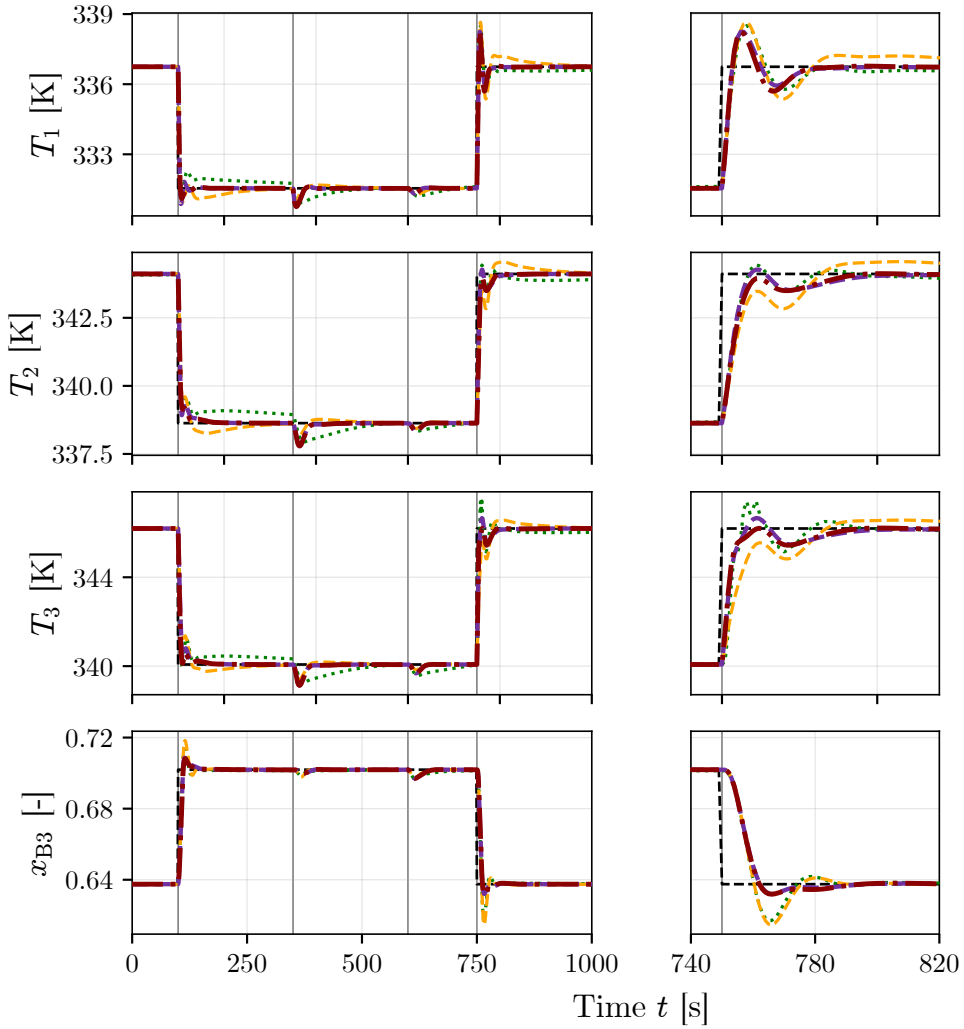


Figure 5: Closed-loop outputs of the simulation scenario. The black dashed line is the setpoint, the dotted green line is the MPC using the N4SID model, the dashed orange line is the MPC using the Koopman model with the linear decoder, the dashed purple line is the proposed formulation linearized at the current state estimate, and the dash-dotted red line is the proposed formulation linearized at the target of the previous sampling instant. The grey vertical lines mark the setpoint changes and the feed disturbances listed in Table 4.

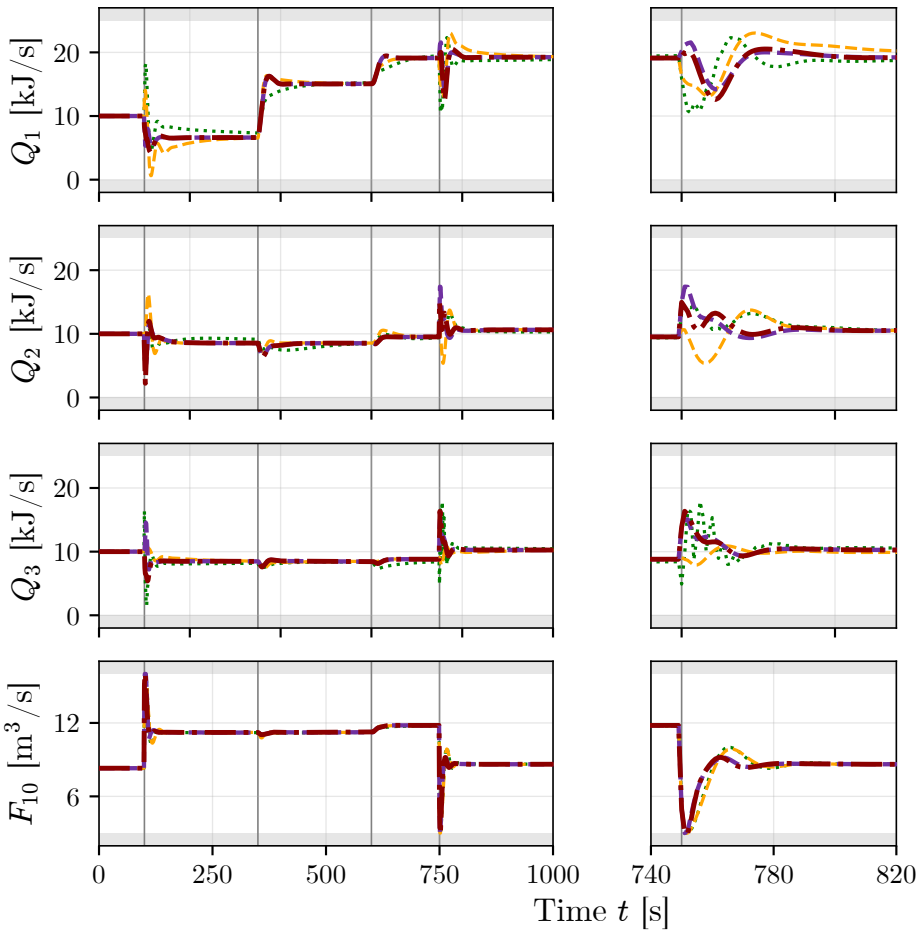


Figure 6: Closed-loop manipulated variables of the simulation scenario. The dotted green line is the MPC using the N4SID model, the dashed orange line is the MPC using the Koopman model with the linear decoder, the dashed purple line is the proposed formulation linearized at the current state estimate, and the dash-dotted red line is the proposed formulation linearized at the target of the previous sampling instant. The grey vertical lines mark the setpoint changes and the feed disturbances listed in Table 4, and the grey bands are the input bounds of Table 1.

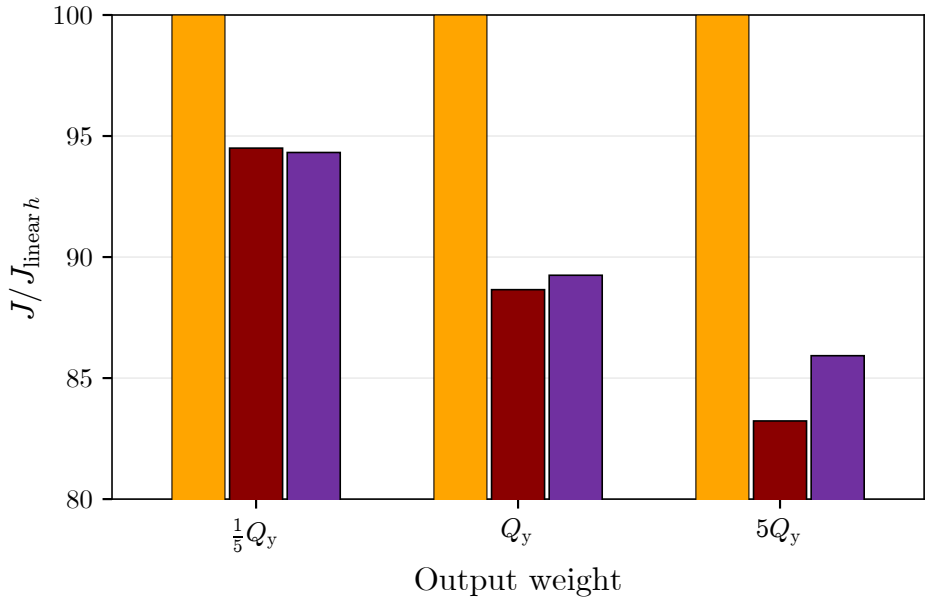


Figure 7: Closed-loop objective J divided by the objective of the Koopman controller with the linear decoder at the same tuning. The orange bars are the linear decoder, the red bars the proposed formulation linearized at the previous target, and the purple bars the proposed formulation linearized at the current state estimate.

method lies in the output map, so when the outputs are more penalized, it has a greater improvement compared to lower penalization of outputs.

8. Discussion

In this section, we interpret the identification results of Sec. 5 together with the closed-loop results of Sec. 7. The identification shows that the Koopman model with the nonlinear decoder, which is the model used by the proposed formulation, is the most accurate of the three models in Table 3. Restricting its output mapping to a matrix increases the summed standardized MAE from 0.21 to 1.21, so that the nonlinear decoder reduces the open-loop prediction error by approximately 83% with respect to the linear one. Moreover, the Koopman model with the linear decoder is even less accurate in open loop than the N4SID model, because the restricted decoder leaves the input–output map of an LTI system, for which subspace identification is the better-tuned tool. On the other hand, the right hyperparameters of the neural networks are harder to find than those of the subspace model. We select them by an automatic hyperparameter search, which is time-consuming. Nevertheless, the identification is carried out in the offline phase, and the benefits that the nonlinear decoder brings to the control loop are worth this effort. Note that the MAE serves only as the identification criterion, and the behavior of the models in the closed loop can, and in general will, differ from their ranking in open loop. This is the case of the Koopman model with the linear decoder, which has a higher open-loop MAE than the N4SID model in Table 3 but attains a lower closed-loop objective in Table 5.

As can be seen in Table 5, all three Koopman controllers attain a lower closed-loop objective J than the MPC using the N4SID model, which represents the industrial benchmark. Furthermore, the proposed formulation also outperforms the Koopman controller with the linear decoder, which represents the state-of-the-art Koopman MPC. Measured relative to the objective of this controller, and not to the N4SID objective of the last column of Table 5, the proposed formulation lowers J by 11.3% and 10.8% when linearized at the previous target and at the current state estimate, respectively. Since the objective accumulates the deviation from the requested grade together with the effort spent on the inputs, a reduction of this size is not merely incremental but significant for the operation of the whole process.

As can be seen in Fig. 5, the two variants of the proposed formulation do not differ much. From our perspective, the linearization at the current state

estimate is more intuitive than the linearization at the target of the previous sampling instant. However, which of the two points is better suited depends on the specific plant, on the requirements of the followed references, and on how far apart the setpoints lie and how nonlinear the dynamics between them is. We present both variants, since they reach a similar performance, and overall they are almost equal near the target. The choice of the linearization point therefore remains an open question, and its assessment on further benchmark processes, with different degrees of nonlinearity and different distances between the setpoints, would clarify which variant is better suited for a given application.

The proposed formulation does not improve the rejection of the feed disturbances. After the colder feed, both proposed variants show the largest temperature deviation of the four controllers, and after the leaner feed the controller with the linear decoder shows the smallest one. Both disturbances act on the plant only and enter the control problem solely through $\hat{d}(t)$, and the two proposed variants respond to them almost identically, which indicates that the linearization point has little influence on the rejection of a disturbance at a fixed operating point. The advantage of the proposed formulation is therefore concentrated in the transitions between operating points, where the nonlinearity of the output mapping matters most.

The sensitivity study of Fig. 7 compares only the Koopman controllers, without the N4SID controller, and its objective is normalized to that of the Koopman controller with the linear decoder at the same tuning. It can be observed that the gap between the proposed formulation and the linear decoder grows as more emphasis is placed on the penalization of the outputs. Since the improvement of the proposed formulation lies in the output mapping, this trend shows its strong side.

9. Conclusion

References

Akiba, T., Sano, S., Yanase, T., Ohta, T., Koyama, M., 2019. Optuna: A next-generation hyperparameter optimization framework, in: Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining, pp. 2623–2631.

Armenise, G., Bacci di Capaci, R., Vaccari, M., Pannocchia, G., 2018. An open-source system identification package for multivariable processes, in:

2018 UKACC 12th International Conference on Control (CONTROL). doi:10.1109/CONTROL.2018.8516791.

Chen, J., Dang, Y., Han, J., 2022. Offset-free model predictive control of a soft manipulator using the koopman operator. Mechatronics 86, 102871.

Drgoňa, J., Tuor, A., Koch, J., Shapiro, M., Vrabie, D., 2023. NeuroMANCER: Neural Modules with Adaptive Nonlinear Constraints and Efficient Regularizations URL: https://github.com/pnnl/neuromancer.

Gurobi Optimization, LLC, 2024. Gurobi Optimizer Reference Manual. URL: https://www.gurobi.com.

Kaiser, E., Kutz, J.N., Brunton, S.L., 2021. Data-driven discovery of koopman eigenfunctions for control. Machine Learning: Science and Technology 2, 035023.

Koopman, B.O., 1931. Hamiltonian systems and transformation in hilbert space. Proceedings of the National Academy of Sciences 17, 315–318.

Korda, M., Mezić, I., 2018. Linear predictors for nonlinear dynamical systems: Koopman operator meets model predictive control. Automatica 93, 149–160.

Li, H., Swartz, C.L.E., 2019. Dynamic real-time optimization of distributed MPC systems using rigorous closed-loop prediction. Computers & Chemical Engineering 122, 356–371. doi:10.1016/j.compchemeng.2018.08.028.

Liu, J., Muñoz de la Peña, D., Christofides, P.D., 2009. Distributed model predictive control of nonlinear process systems. AIChE Journal 55, 1171–1184.

Lusch, B., Kutz, J.N., Brunton, S.L., 2018. Deep learning for universal linear embeddings of nonlinear dynamics. Nature communications 9, 4950.

Mezić, I., 2005. Spectral properties of dynamical systems, model reduction and decompositions. Nonlinear Dynamics 41, 309–325.

Pan, C., Li, Y., Dong, L., 2024. Offset-free koopman model predictive control of thermal comfort regulation for a variable refrigerant flow-dedicated outdoor air system-combined system. Journal of Dynamic Systems, Measurement, and Control 146, 051002.

Pannocchia, G., Rawlings, J.B., 2003. Disturbance models for offset-free model-predictive control. AIChE journal 49, 426–437.

Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., Desmaison, A., Köpf, A., Yang, E., DeVito, Z., Raison, M., Tejani, A., Chilamkurthy, S., Steiner, B., Fang, L., Bai, J., Chintala, S., 2019. Pytorch: An imperative style, high-performance deep learning library, in: Advances in Neural Information Processing Systems 32, pp. 8024–8035.

Rowley, C.W., Mezić, I., Bagheri, S., Schlatter, P., Henningson, D.S., 2009. Spectral analysis of nonlinear flows. Journal of fluid mechanics 641, 115–127.

Van Overschee, P., De Moor, B., 2012. Subspace identification for linear systems: Theory—Implementation—Applications. Springer Science & Business Media.

Williams, M.O., Kevrekidis, I.G., Rowley, C.W., 2015. A data–driven approximation of the koopman operator: Extending dynamic mode decomposition. Journal of Nonlinear Science 25, 1307–1346.